



U-15 長野 プログラミング コンテスト



講習の内容

- U-15 長野プログラミングコンテストについて
- プログラミングの基本学習
- 補足資料



U-15 長野
プログラミング
コンテスト

U15長野プログラミングコンテストについて



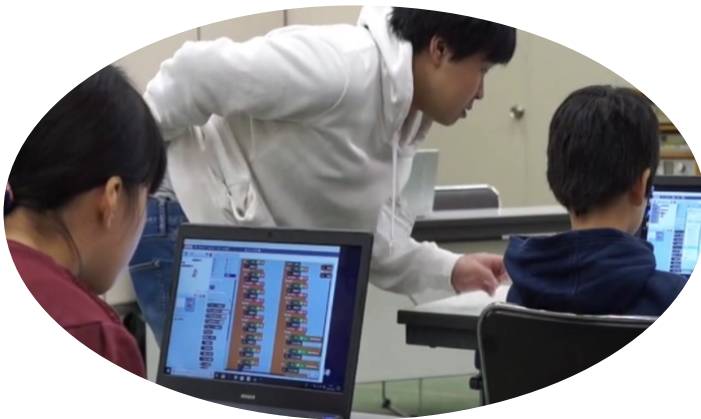
U-15長野
プログラミング
コンテスト

U-15長野プログラミングコンテストとは

2018年から長野市で開催されている15歳以下を対象としたプログラミングのコンテストです。

対戦形式のゲーム（CHaser）で勝敗を競います。

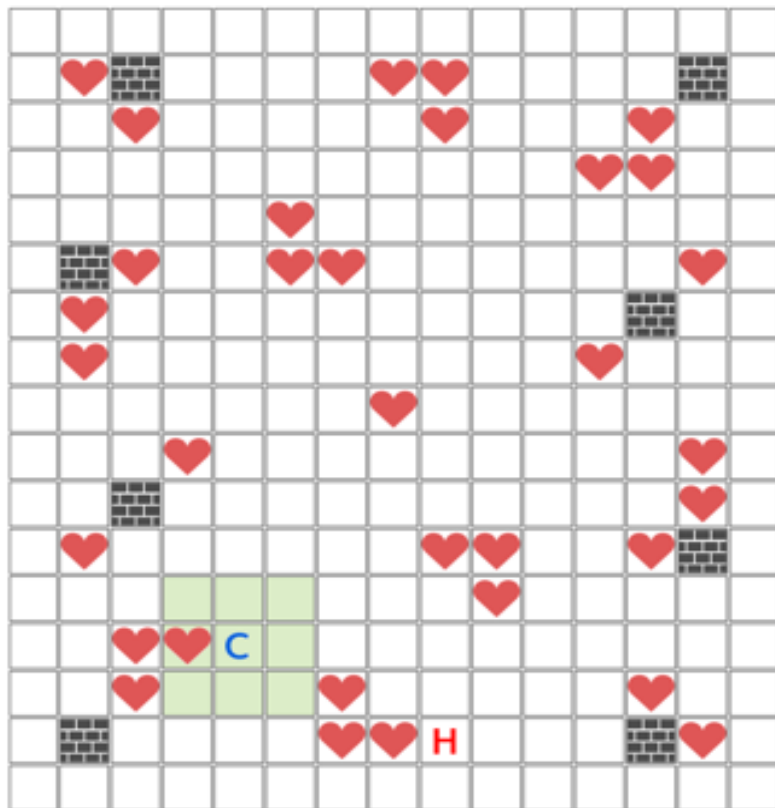
事前講習会は、信州大学工学部・長野工業高等専門学校・長野工業高等学校の学生が講師となり、プログラミングを教えます。





コンテストの内容

CHaser (チェイサー)

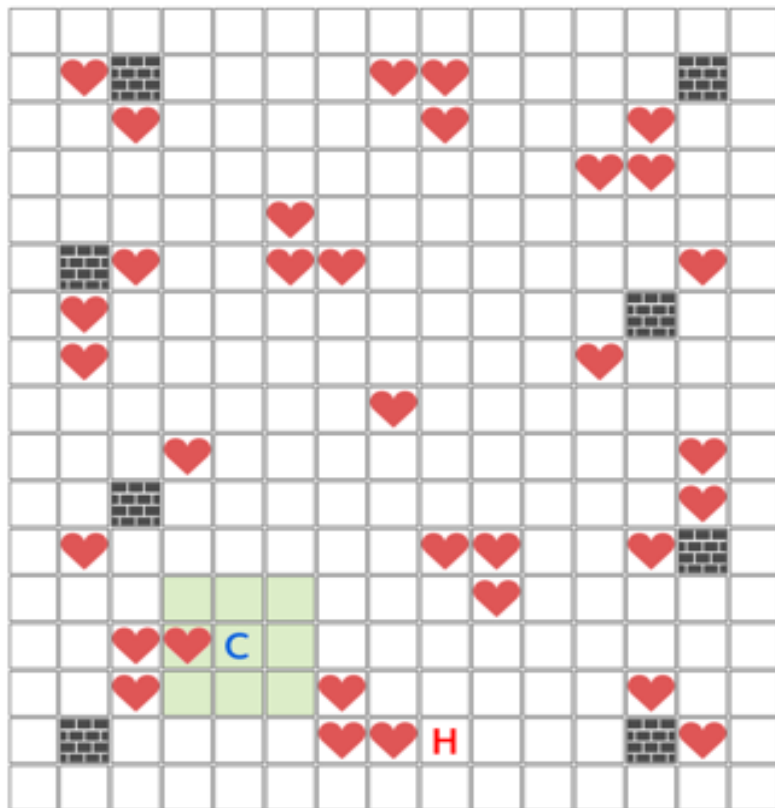


15×17マスの盤面上で、
2人のプレイヤーが1対1で対戦する
ゲーム。

プレイヤーCoolとプレイヤーHotが
互いに攻撃し合いながら、
アイテムを取り合うターン制ゲーム。
(所定のターン数終了まで)

コンテストの内容

CHaser (チェイサー)



命令は全部で **4 種類**

- ・ 1マス移動 (walk)
- ・ 周囲確認① (look)
- ・ 周囲確認② (search)
- ・ ブロックを置く (put)

	...	プレイヤー 1
	...	プレイヤー 2
	...	アイテム
	...	ブロック



コンテストの内容

CHaser (チェイサー)

(ゲーム動画)



基本的な使い方

Googleアカウント作成

事前にご案内しました通りGoogleアカウントが必要です
作っていない場合はお近くの講師にご相談ください

<https://www.google.com/intl/ja/account/about/>

A screenshot of the Google Account creation page in Japanese. The page has a dark background. At the top left is the Google logo. Below it, the text 'Google アカウントを作成' (Create Google Account) is displayed, followed by the instruction '名前を入力してください' (Please enter your name). To the right of the text are two input fields: the top one is labeled '姓 (省略可)' (Surname, optional) and the bottom one is labeled '名' (Given name). At the bottom right of the form is a blue button labeled '次へ' (Next). At the very bottom of the page, there is a footer with the text '日本語' (Japanese) on the left, and 'ヘルプ' (Help), 'プライバシー' (Privacy), and '規約' (Terms) on the right.



基本的な使い方

CHaserを実行する

ブラウザを使って以下のサイトにアクセスしてください

<https://colab.research.google.com/drive/1oGRrTwLN2Gnr0Ywyhu3qPIBBXy2ccObd?usp=sharing>

The screenshot shows a Google Colab notebook with the following content:

- 目次**
 - 対戦ゲーム CHaser (チェイサー) とは
 - 勝利条件
 - 命令と動作例
 - サンプルプログラムの実行方法
 - + セクション
- サンプルプログラムの実行方法**
 - 下記の(1)と(2)を順番に実行してください。
 - 一度(1)を実行しておけば、その後はランタイムから切断しない限り、(2)だけ実行すれば繰り返し実行できます。
 - 編集した内容を保存したい場合には、このノートブックをコピーしてご自身のアカウントで使ってください。
 - (1) 下記のセルを実行して、プログラムの実行に必要な環境を構築します。

```
[ ] !wget -Nq http://procon.nagano-it.org/lib/chaser.py && echo "ok." || echo "error."
!pip3 install python-socketio[client]
```
 - (2) 下記のセルを実行して対戦プログラムを実行します。
 - Colabを実行するタイミングによっては、動作に時間がかかりタイムアウトで対戦が終了します。
 - その場合は、再度セルを実行してみてください。
 - (3) 使用可能なルーム一覧
 - 練習01 全てのアイテムを取れ！ : pvt_prac01
 - 練習02 相手にPUTせよ！ : pvt_prac02
 - 練習03 アイテムを3つ以上取れ！ : pvt_prac03

The code cell for step (2) contains the following Python code:

```
[ ] import chaser
import time

target = chaser.Chaser()
target.set_name("プレーヤ1")
room = "pvt_prac01"
target.set_room(room)
target.connect_server()
target.show()

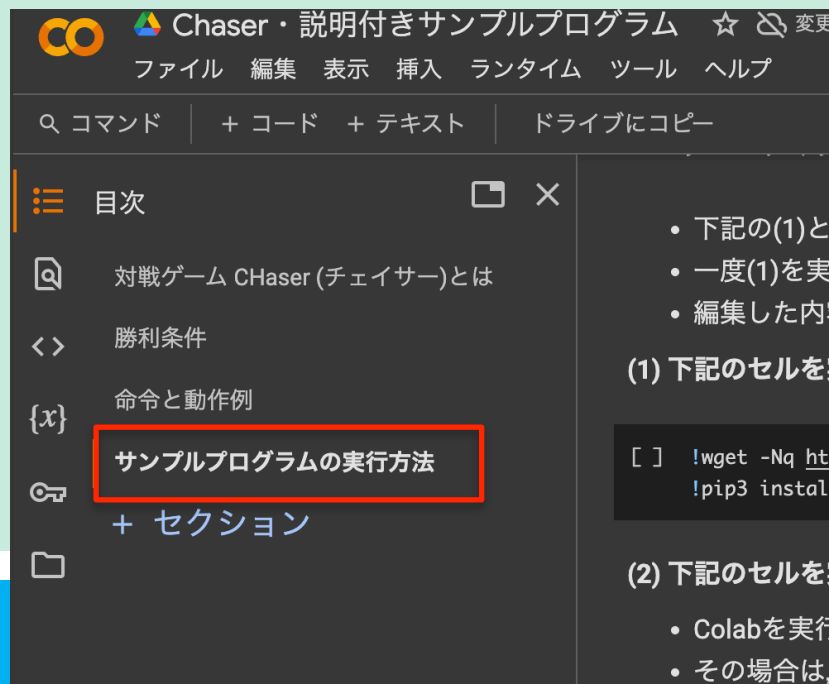
while target.is_connected:
    value = target.get_ready()
    result = target.look_down()
    time.sleep(0.1)
```



基本的な使い方

CHaserを実行する

「目次」 → 「サンプルプログラムの実行方法」を選択





基本的な使い方

CHaserを実行する

以下の(1)のセルの実行ボタン (▶)をクリックします

(1) 下記のセルを実行して、プログラムの実行に必要な環境を構築します。



```
!wget -Nq http://procon.nagano-it.org/lib/chaser.py && echo "ok." || echo "err  
!pip3 install "python-socketio[client]"
```



基本的な使い方

CHaserを実行する

警告メッセージは「このまま実行」をクリックしてください
完了まで少し時間がかかります

クの作成者は kby@cs.shinshu-u.ac.jp です。Google に保存されているデータ
求められたり、他のセッションからデータや認証情報が読み取られたりする
。このノートブックを実行する前にソースコードをご確認ください。ご不明な
このノートブックの作成者（kby@cs.shinshu-u.ac.jp）にお問い合わせくださ

キャンセル

このまま実行



基本的な使い方

CHaserを実行する

以下の(2)のセルの実行ボタン (▶)をクリックします

(2) 下記のセルを実行して対戦プログラムを実行します。

- Colabを実行するタイミングによっては、動作に時間がかかりタイムアウトで
- その場合は、再度セルを実行してみてください。



```
import chaser
import time

target = chaser.Chaser()
target.set_name("プレイヤー1")
room = "pvt_prac01"
target.set_room(room)
target.connect_server()
target.show()
```



基本的な使い方

CHaserを実行する

ゲームが開始されれば成功です。

room_onetime_練習01 全てのアイテムを取れ!

残りターン数
77

cool プレーヤ1
0 point

hot cpu
0 point

残りアイテム数
76

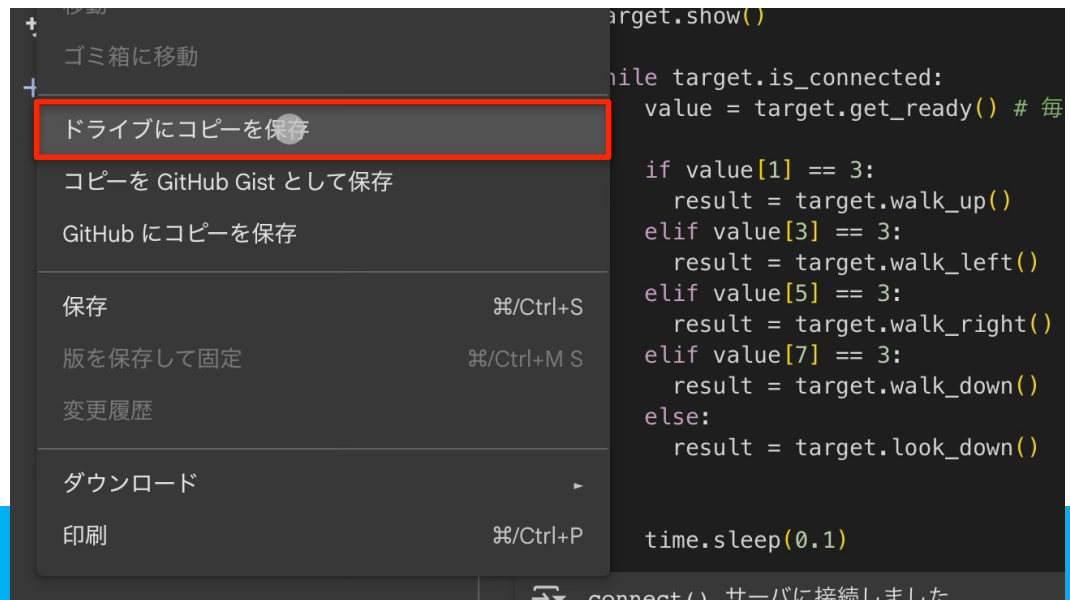


基本的な使い方

コードを保存する

ブラウザを閉じると、コードは保存されません。
次の方法でバックアップを行います

1. 左上の「ファイル」を
クリックします
2. 「ドライブにコピーを保存」を
クリックします





基本的な使い方

コードを保存する

ブラウザを閉じると、コードは保存されません。
次の方法でバックアップを行います

3. 「このサイトではポップアップウィンドウが開きます」というメッセージが出ます

(iPadの場合)

「許可」をクリックします





基本的な使い方

コードを保存する

自分のGoogleドライブに環境がコピーされます

Chaser - 説明付きサンプルプログラム のコピー ☆ ☰

ファイル 編集 表示 挿入 ランタイム ツール ヘルプ

🔍 コマンド + コード + テキスト

☰ 目次

- 対戦ゲーム CHaser (チェイサー)とは
- <> 勝利条件
- 🔑 命令と動作例
- 📄 サンプルプログラムの実行方法
- 📁 + セクション

対戦ゲーム CHaser (チェイサー)とは

- 2人のプレイヤーが15×17マスの盤面上で1対1で対戦するボードゲーム
- プレイヤーCoolとプレイヤーHotがターン制で行動する
- 1ターンで実行できる行動は1つ
- 行動の種類
 - 1マス移動 (walk)
 - 周囲確認① (look)
 - 周囲確認② (search)
 - ブロックを置く (put)
- ゲーム画面とアイコンの意味
 - C: 先手プレイヤー
 - H: 後手プレイヤー
 - ♥: アイテム
 - ■: ブロック



基本的な使い方

コードを保存する

次回以降に再度開けるようにブックマークしておきましょう！

右上の共有アイコン  をクリックして、
「ブックマークに追加」または
「お気に入りに追加」をクリックします





基本的な使い方

基本操作について

基本操作についてはColab上にも説明があります

分からなくなったら
参照してみてください

The screenshot shows a Google Colab notebook interface. The title bar reads "Chaser - 説明付きサンプルプログラムのコピー" (Chaser - Copy of sample program with explanation). Below the title bar is a menu bar with "ファイル", "編集", "表示", "挿入", "ランタイム", "ツール", and "ヘルプ". A search bar contains "コマンド" and buttons for "+ コード" and "+ テキスト". The left sidebar shows a table of contents with sections: "対戦ゲーム CHaser (チェイサー)とは", "勝利条件", "命令と動作例", "サンプルプログラムの実行方法", and "+ セクション". The main content area is titled "対戦ゲーム CHaser (チェイサー)とは" and contains a bulleted list of game rules and a small grid diagram.

対戦ゲーム CHaser (チェイサー)とは

- 2人のプレイヤーが15×17マスの盤面上で1対1で対戦するボードゲーム
- プレイヤーCoolとプレイヤーHotがターン制で行動する
- 1ターンで実行できる行動は1つ
- 行動の種類
 - 1マス移動 (walk)
 - 周囲確認① (look)
 - 周囲確認② (search)
 - ブロックを置く (put)
- ゲーム画面とアイコンの意味
 - C: 先手プレイヤー
 - H: 後手プレイヤー
 - ♥: アイテム
 - ■: ブロック



U-15 長野
プログラミング
コンテスト

Python プログラミングの基本学習



命令の種類と実行順序

命令の種類（待機）

- ・ 待機する： `target.get_ready`
 - 自分のターンが来るのを待つ。
 - 自分のターンが来たら、周囲のマス情報を取得する。



命令の種類と実行順序

命令の種類（行動）

- ・ 移動する : `walk` (`walk_up`, `walk_down`, `walk_right`, `walk_left`)
- ・ 探す : `search` (`search_up`, `search_down`, `search_right`, `search_left`)
- ・ 探す : `look` (`look_up`, `look_down`, `look_right`, `look_left`)
- ・ ブロックを置く : `put` (`put_up`, `put_down`, `put_right`, `put_left`)



命令の種類と実行順序

命令の実行順序（例）

待機命令

行動命令

1 ターン目

待機命令

行動命令

2 ターン目

待機命令

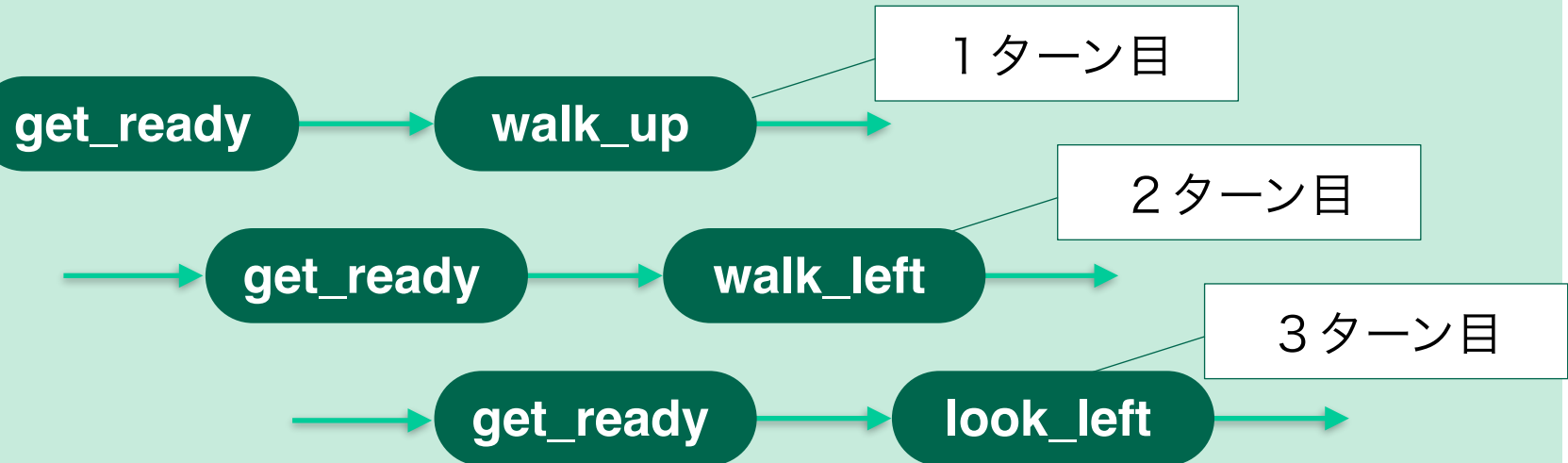
行動命令

3 ターン目



命令の種類と実行順序

命令の実行順序（例）





コードを書いてみよう

ターンの開始

行動開始前に、必ずget_ready命令を実行します。

この命令が実行されると、

周辺情報がvalue配列（＊）に格納されます。

アイテムがある → value[0]は 3

ブロックがある → value[0]は 2

相手プレイヤーがいる → value[0]は 1

何もない → value[0]は 0

＊「配列」は後述します。

value[0]

0	1	2
3	C	5
6	7	8



コードを書いてみよう

共通の書き方（行動系）

- 命令の書き方

result = target.**[実行する命令]**();

※ 実行結果がresultという配列に格納されます。

- walkの場合は、移動後の周辺情報
- lookの場合は、指定方向3×3マスの情報



コードを書いてみよう

walk命令の種類

walk命令は 4 種類あります。

命 令	動 き
walk_up	上に 1 マス移動する
walk_right	右に 1 マス移動する
walk_left	左に 1 マス移動する
walk_down	下に 1 マス移動する



walk

例. walk_up (上への移動)

<書き方>

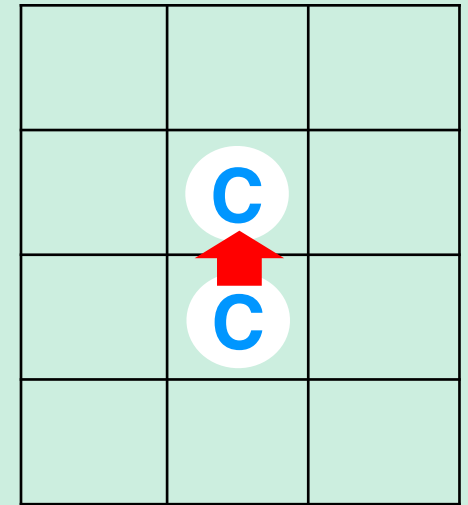
```
result = target.walk_up();
```

<動き>

1. 上のマスに移動します。
2. 移動した後の自分の周辺情報が

resultに取得されます。

(命令が1回実行された扱いになり、このターンには他の命令は実行できません)





walk命令を実行した後のresult

移動した後の周囲の情報がresultに入る。

例. result[0] には移動した後の
マスの左上の情報が入っている。

アイテムがある → result[0]は 3

ブロックがある → result[0]は 2

相手プレイヤーがいる → result[0]は 1

何もない → result[0]は 0

result[0]		
0	1	2
3	C	5
6	C	8



練習問題

制限時間 20分

walk命令を実行して、マップ上を移動してみよう

＊どの方向に動いても構いません

＊壁にぶつかっても良いです



コードを書いてみよう

命令の種類

look命令は 4 種類あります。

命 令	動 き
look_up	上3×3マスの情報を取得する
look_right	右3×3マスの情報を取得する
look_left	左3×3マスの情報を取得する
look_down	下3×3マスの情報を取得する



look

例. look_up (上を調べる)

<書き方>

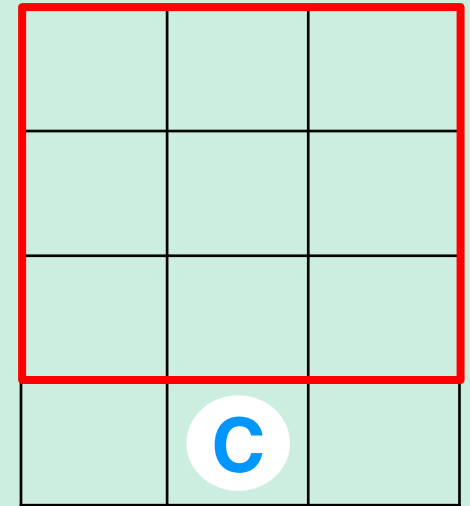
```
result = target.look_up();
```

<動き>

1. 上3×3マスの情報を取得します。
2. 取得した計9マスの情報が

resultに取得されます。

(命令が1回実行された扱いになり、このターンには他の命令は実行できません)





look命令を実行した後のresult

lookした方向3×3マスの情報がresultに入る。

例. result[7] には頭上のマスの
情報が入っている。

アイテムがある → result[7]は 3
ブロックがある → result[7]は 2
相手プレイヤーがいる → result[7]は 1
何もない → result[7]は 0

result[0]

0	1	2
3	4	5
6	7	8
	C	



コードを書いてみよう

命令の種類

search命令は4種類あります。

命 令	動 き
search_up	上に直線上9マスの情報を取得する
search_right	右に直線上9マスの情報を取得する
search_left	左に直線上9マスの情報を取得する
search_down	下に直線上9マスの情報を取得する



search

例. search_left (左を調べる)

<書き方>

```
result = target.search_left();
```

<動き>

1. 左の直線状9マスの情報を取得します。
2. 取得した計9マスの情報がresultに取得されます。

(命令が1回実行された扱いになり、このターンには他の命令は実行できません)





search命令を実行した後のresult

searchした方向の直線状9マスの情報がresultに入る。

例. result[0] には左のマスに入っている。

アイテムがある → result[0]は 3

ブロックがある → result[0]は 2

相手プレイヤーがいる → result[0]は 1

何もない → result[0]は 0





コードを書いてみよう

命令の種類

put命令は 4 種類あります。

命 令	動 き
put_up	上にブロックを置く
put_right	右にブロックを置く
put_left	左にブロックを置く
put_down	下にブロックを置く



Put

例. put_up (上へブロックを置く)

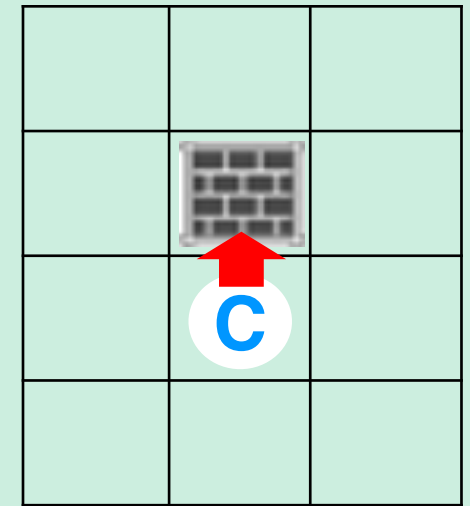
<書き方>

```
result = target.put_up();
```

<動き>

1. 上のマスにブロックを置きます。
2. ブロックを置いた後の自分の周辺情報が
resultに取得されます。

(命令が1回実行された扱いになり、このターンには他の命令は実行できません)





put命令を実行した後のresult

ブロックを置いた後の
周囲の情報がresultに入る。

例. result[1] には頭上のマス
の情報が
入っている。

put命令で頭上にブロックを置いた後なので
result[1]は 2

result[1]		
0		2
3	C	5
6	7	8



コードを書いてみよう

条件分岐の構文 (if)

If構文を使った分岐の例

```
# resultの要素に応じて移動方向を決定する
if result[1] == 0:
    result = client.walk_up()
elif result[3] == 0:
    result = client.walk_left()
elif result[5] == 0:
    result = client.walk_right()
elif result[7] == 0:
    result = client.walk_down()
else:
    result = client.look_right()
```



コードを書いてみよう

if構文

if 条件式①:

条件式①が成立したときに実行される命令

elif 条件式②:

条件式②が成立したときに実行される命令

elif 条件式③:

条件式③が成立したときに実行される命令

else:

それ以外のときに実行される命令



コードを書いてみよう

if構文

- 1個目は必ず if。
- elif は if または 他の elif の後にのみ使うことができる。
if または elif の条件式が成立した場合は、それ以降の分岐には入らない。
- elif は何個あっても良い。
- else を使う場合は、一番最後に 1 回のみ使うことができる。



コードを書いてみよう

if構文の練習①

自分の周りにアイテムがあったら取得して、アイテムが無ければ
上方向にLookするコードを書いてみよう。



コードを書いてみよう

条件式で使う比較演算子

演算子	意 味
==	左右が同じものである
!=	左右が違うものである
<	右のものは左のものよりも大きい
<=	右のものは左のもの以上
>	左のものは右のものよりも大きい
>=	左のものは右のもの以上



コードを書いてみよう

if構文のサンプル③

条件式を2つ以上記述するif構文のサンプル

複数の条件式を

「and」（意味：かつ）

「or」（意味：または）

で繋ぐと、複数の条件を
判定できるようになります。

```
# resultの要素に応じて移動方向を決定する
if result[1] != 1 and result[1] != 2:
    result = client.walk_up()
elif result[3] != 1 and result[3] != 2:
    result = client.walk_left()
elif result[5] != 1 and result[5] != 2:
    result = client.walk_right()
elif result[7] != 1 and result[7] != 2:
    result = client.walk_down()
else:
    result = client.look_right()
```



コードを書いてみよう

条件分岐の構文 (match)

match構文 (python 3.10以降) を使った分岐の例

```
match result[1]:  
    case 1:  
        result = client.put_up()  
    case 3:  
        result = client.walk_up()  
    case 0 | 2:  
        result = client.look_right()
```



練習問題

制限時間 30分

これまでの内容を活用して、練習01～04に挑戦してみましょう。

ルームIDは以下を参考にしてください。

(練習問題のルーム)

(コードの変更箇所)

(3) 使用可能なルーム一覧

- 練習01 全てのアイテムを取れ！ : pvt_prac01
- 練習02 相手にPUTせよ！ : pvt_prac02
- 練習03 アイテムを3つ以上取れ！ : pvt_prac03
- 練習04 最後まで生き残れ！ : pvt_prac04

```
target = chaser.Chaser()  
target.set_name("プレイヤー1")  
room = "pvt_prac01"  
target.set_room(room)  
target.connect_server()  
target.show()
```



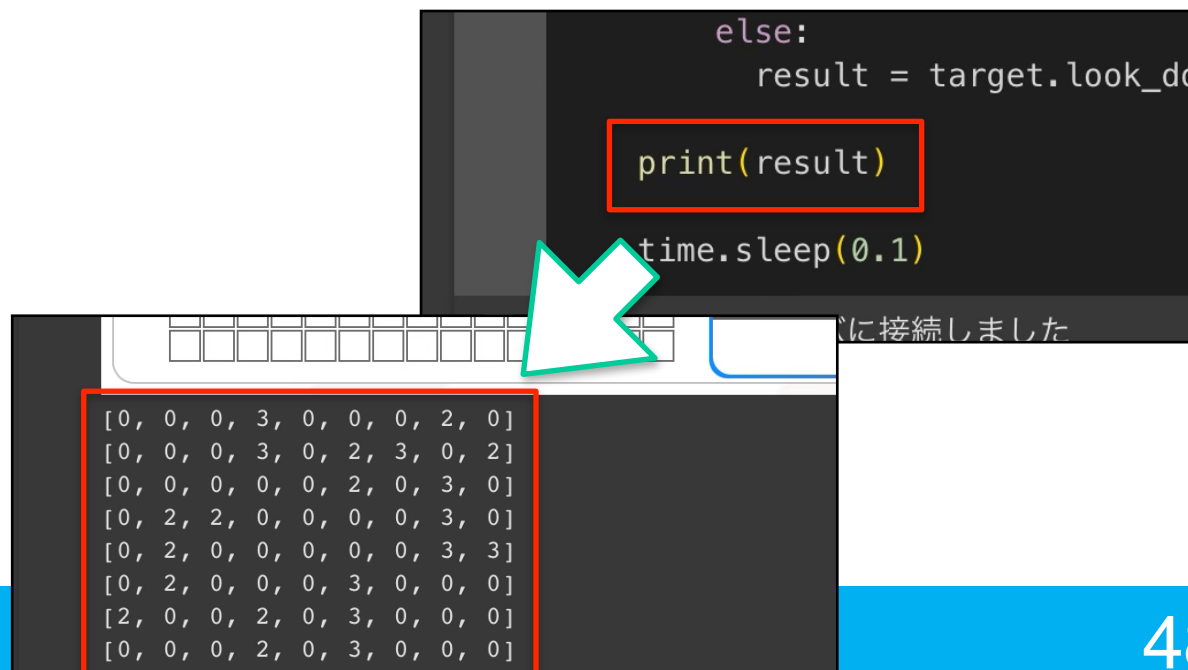
コードを書いてみよう

思い通りに動かない時は

変数や配列に何が入っているか確認してみましょう

例. `print(result)`

`result`に格納されている
データが出力されます





コードを書いてみよう

自分でマップを作りたい時は

次のURLより、マップを作成することができます

<https://procon.nagano-it.org/mapeditor/>

ただし、以下の注意点があります

- 作成したマップは使用後に自動的に削除されます
 - 使用されなかった場合には、一定時間(10分)経過後に
 - 別のマップがアップされる
 - あいことばルーム(任意名の対戦ルーム)が作られる
- タイミングで削除されます



コードを書いてみよう

自分でマップを作りたい時は

マップ作成

アップロードパスワード
「procon15_nagano」

Procon Room Editor

マップサイズX:

マップサイズY:

マップ生成方法: 手動

ルーム名:

ターン数:

CPUとの対戦? ☐

ダウンロード 設定をロード

アップロード先URL:

パスワード:

サーバにアップロード



U-15 長野
プログラミング
コンテスト

補 足 資 料 (大会ルール)



ルール説明

基本ルール

予選大会のルール①

- 運営が用意したボットプログラムと対戦する。
プレイヤーは必ず先攻。
- 1人2回、対戦を行う。
1回目と2回目のマップは変わる。
- ターン数は100ターン。



ルール説明

基本ルール

予選大会のルール②

- ゲーム終了時、得点を計算する
 - ✓ 取得したアイテム数 × 3 点
 - ✓ PUTで勝った場合、残りターン数を得点に加算
 - ✓ PUTで負けた場合、または自滅した場合
残りターン数を点数から減算



ルール説明

基本ルール

マップの制約

- 全部で3パターンのマップがある（「競技部門運営細則」参照）。
- プレイヤー、アイテム、ブロックの配置は点対称
- アイテムを取ったとき、元々いたマスはブロックに変わる。
- 開始時、上下の9マス目にはブロックは存在しない。



ルール説明

基本ルール

決勝大会のルール①

- 予選大会を勝ち上がった上位4名（※）がトーナメント形式でプレイヤー同士が対戦する。
- 先行／後攻を入れ替えて同じマップで2回対戦する。
- ターン数は100ターン。

※ 昨年度はBlockly言語の部：4名、C#言語の部：4名が決勝に進出。



ルール説明

基本ルール

決勝大会のルール②

- 勝った回数が多いプレイヤーの勝利
- 勝った回数が同じである場合、
引き分け処理を行う（「競技部門運営細則」参照）
- それでも決着がつかない場合、
マップを変更して再戦を行う



U-15 長野
プログラミング
コンテスト

お問合せ先

ご不明な点は以下までお問合せ下さい。

お問合せ先

U-15 長野プログラミングコンテスト実行委員会事務局

長野商工会議所内 TEL：026-227-2428 担当：小林 万里子